

Spike: 8**Title:** Soldier on Patrol**Author:** Pattarapol Tantechasa, 103883220**Goals / deliverables:****Expected Outcomes (Baseline Task)**

1. **Executable Code:** A working "soldier on patrol" simulation that clearly demonstrates a layered FSM architecture managing both high-level modes (Patrol/Attack) and low-level step behaviours.
2. **Spike Report:** A written analytical report detailing your FSM architecture, the specific rules dictating state transitions, and an evaluation of the system's effectiveness. This report must be saved as a .pdf.

Expected Outcomes (from extensions)

1. **Extended Codebase:** Working simulation code that successfully demonstrates the new modes, multi-agent interactions, or environmental awareness without breaking the base FSM logic.
2. **Extended Report Analysis:** An expanded section in your Spike Report PDF that details the mathematical or logical hurdles encountered when implementing these advanced state transitions and regional behaviours.

Technologies, Tools, and Resources used:

- Claude: brainstorm ideas, explain code and debugs.
- Visual Studio Code IDE
- Conda: Python environment manager

Tasks undertaken:

1. Download and Install **Visual Studio Code**
2. Install **Python** and **Pyglet** on your Machine
3. Open the main project downloaded
4. Open terminal and run cd command : **cd Portfolio-Tasks/Portfolio-Task-16-Spike-Soldier-On-Patrol**
5. For baseline version : **cd soldier-baseline**
6. For extended version : **cd soldier-extensions**
7. Run : **python main.py**

Design Phase: Soldier on Patrol

Overview

The goal of this spike was to bridge the gap between simple steering behaviours and structured decision-making by implementing a layered Finite State Machine (FSM). Rather than having a single agent that directly maps inputs to movement, I designed a two-layer architecture where high-level states decide what the agent should do, and low-level steering behaviours handle how to do it. I used the path-following steering codebase from Task 12 as the movement foundation and the projectile combat logic from Task 15 as the attack foundation, then built the FSM system on top of both.

Simulation Setup

I created a soldier agent that starts by following a randomly generated looped waypoint path around the screen. A separate Enemy class represents stationary red circle targets that can be spawned at random positions by pressing E. Each enemy has 100 health points and a visible health bar that depletes as projectiles land. Projectiles are simple yellow circles that travel in a straight line at 300 px/s and deal 25 damage on contact, destroying an enemy in four hits.

Each entity tracks:

1. **pos** — current world position as a Vector2D
2. **vel** — current velocity (agent only)
3. **health / alive** — combat state (enemy only)

Layered FSM Architecture

The core concept of this spike is decoupling the agent's decision logic from its movement logic using the State design pattern. I created a base State class with three methods — **enter()**, **execute()**, and **exit()** — and two concrete state classes that inherit from it.

The Agent class holds a `current_state` reference and a `change_state()` method. Every frame, `calculate()` simply calls `current_state.execute(agent, delta)` and returns whatever steering force the state produces. This means the agent itself has no mode-checking logic — all decisions live inside the state objects.

```
Agent.calculate(delta)
├─ current_state.execute(agent, delta)
│   ├─ PatrolState → returns follow_path() steering force
│   └─ AttackState → returns Vector2D(0, 0), fires projectiles via
timer
```

PatrolState

PatrolState represents the macroscopic Patrol mode. Each frame, its `execute()` method first checks the environment for threats by calling `agent.detect_enemy()`, which scans the world's enemy list and returns True if any live enemy is **within 200 px**. If a threat is detected, the agent immediately calls `change_state(AttackState())` — this is the transition rule. If no threat exists, the state delegates movement to the low-level `follow_path()` steering behaviour, which sequences through the waypoints using `seek()` and `arrive()`. The soldier is drawn in orange while in this state.

AttackState

AttackState represents the macroscopic Attack mode. On entry, it zeroes the agent's velocity immediately so the soldier stops in place, and resets a shoot cooldown to zero so the first shot fires without delay. Each frame, `execute()` first checks whether any enemies remain in range — if not, it transitions back to PatrolState, allowing the soldier to resume its patrol from the same waypoint it left. If enemies are still present, it decrements a cooldown timer and fires a Projectile at the nearest live enemy when the timer expires, then resets it to 1.5 seconds. The soldier is drawn in red while in this state.

The transition rules are:

From	To / Condition
PatrolState	AttackState — A live enemy is within 200 px
AttackState	PatrolState — No live enemies remain within 200 px

A key design decision was zeroing the agent's velocity on `AttackState.enter()` rather than just returning zero force. Returning zero force only means zero acceleration — the agent would continue drifting at its current speed due to momentum. Zeroing velocity directly produces an immediate stop, which is the correct soldier behaviour.

Outcome

The baseline simulation successfully demonstrates a working layered FSM. The soldier patrols autonomously, reacts to spawned enemies by stopping and engaging, and resumes patrol cleanly after all enemies are eliminated. The state name and enemy count are displayed live in the HUD, making the transitions visible and verifiable during testing. The architecture cleanly separates the decision layer from the behaviour layer, meaning new states can be added without modifying the existing ones.

Extension 1: Enemy Shoots Back

I gave each enemy an `update()` method that fires a `Projectile` at the nearest visible soldier every 2 seconds, but only if a soldier is within a 250 px detection range. Enemy projectiles deal 10 damage (versus the soldier's 25), so combat is slightly asymmetric — soldiers are more dangerous per shot but fewer in number.

To support this, I added a health system to the `Agent` class: `health = 100`, `max_health = 100`, and a `take_damage(amount)` method that clamps health at zero. The existing `Projectile` class was extended to accept a damage parameter so both soldiers and enemies can share the same projectile type with different damage values. The HUD now shows each soldier's current health in brackets (e.g. *S1: PatrolState [87hp]*).

Extension 2: FleeState and HealingState

With soldiers now taking damage, I needed a way for them to respond to low health. I added two new states, both using the same transition threshold constant: soldiers enter **low-health behaviour below 30 hp** and **leave it only when health returns above 80 hp**. This gap between entry and exit thresholds is called hysteresis — it prevents the soldier from rapidly bouncing back into combat the moment they receive a single heal tick.

FleeState activates when health is critical and an enemy is in detection range. The soldier turns purple and applies the `flee()` steering behaviour away from the nearest enemy. Once no enemies remain in range, it transitions to `HealingState` if health is still low and a station exists, otherwise returns to `PatrolState`.

HealingState navigates the soldier to a fixed-position `HealingStation` in the bottom-left corner using `arrive()`, then recovers 20 hp per second while the soldier is within 20 px of the station. Enemy presence overrides healing — a soldier that detects a threat while recovering will flee or attack depending on current health. The `HealingStation` is a semi-transparent green circle with a label, created once in the `World` constructor and persisting for the entire simulation.³

Extension 3: Ammo and Reload

I added a clip-based ammo system inside `AttackState`. The soldier carries a magazine of 6 rounds, firing once per second. When the magazine empties, the state enters a 2-second reload phase during which no shots are fired. After reloading, the magazine refills and shooting resumes automatically.

All ammo state (ammo, reloading, reload_timer) is stored on the state object itself, not on the agent, keeping the agent class clean. The `status()` method on `AttackState` returns a string — either *[Ammo 4/6]* or *[Reloading 1.5s]*. Which the HUD polls every frame and displays next to the soldier's state name.

Extension 4: Multiple Agents

I spawned three soldiers instead of one by adding a loop in `game.py`. Each agent is initialised independently with its own randomised patrol path and its own FSM instance. Because all FSM state (ammo, reload timer, health) is held inside the agent or the state object, there is no shared mutable data between agents, three soldiers running simultaneously requires no architectural changes.

The HUD lists all three soldiers on separate lines, each showing state, ammo status, and health.

Full Transition Table (Extensions)

The priority hierarchy, **Flee > Attack > Heal > Patrol** — is enforced by checking the higher-priority conditions first in each state's `execute()` method.

From	To	Condition
PatrolState	FleeState	Health < 30 hp AND enemy in range
PatrolState	HealingState	Health < 30 hp AND no enemy in range
PatrolState	AttackState	Enemy in range (health ≥ 30 hp)
AttackState	FleeState	Health drops below 30 hp
AttackState	PatrolState	No enemies remain in range
FleeState	HealingState	No enemies in range AND health < 30 hp
FleeState	PatrolState	No enemies in range AND health ≥ 30 hp
HealingState	FleeState	Enemy detected AND health < 30 hp
HealingState	AttackState	Enemy detected AND health ≥ 30 hp
HealingState	PatrolState	Health reaches 80 hp (hysteresis exit)

Outcome

The extensions demonstrate how cleanly new behaviour can be layered onto an existing FSM architecture. Each extension added new state logic or modified an existing state without touching the other states or the agent's core update loop.

The most interesting design insight from this task is the role of hysteresis in preventing state thrashing — without the 30/80 hp gap, a soldier standing at the edge of a heal zone while taking chip damage would transition between HealingState and FleeState every frame. The multiple-agent extension also confirmed that the state design pattern naturally provides instance isolation: three soldiers sharing the same state classes but maintaining completely separate runtime state with no extra effort.

Conclusion

This spike showed that a layered FSM is a practical and straightforward way to give game agents believable behaviour. By separating what to do (the states) from how to move (the steering behaviours), the system stayed easy to extend, each new feature slotted in without breaking anything already working.

The biggest takeaway is that small design decisions matter a lot. Zeroing velocity on state entry stopped soldiers from sliding when they should stand still. Hysteresis stopped them from flickering between states. These are simple fixes, but without them the agents would feel broken even if the logic was technically correct.

Overall, the FSM pattern is well suited to character AI where behaviour is clear-cut and predictable. For more complex scenarios like an agent that needs to weigh multiple goals at once — it would likely need to be extended with a priority or utility system on top.